

Aula 02 - 24/02/2026

Preparação do ambiente de Desenvolvimento

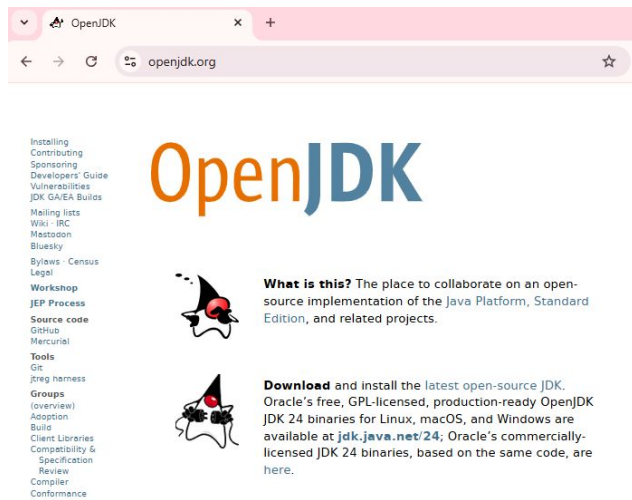
Sumário

1. Java SE e kits de desenvolvimento
 - a. OpenJDK
 - b. Oracle JDK
 - c. GraalVM
2. Editores de código e IDEs
 - a. IntelliJ, VSCode, Eclipse e Netbeans
3. Gerenciadores de dependências
 - a. Maven, Gradle e Ant
4. Agentes de Codificação
 - a. GEMINI CLI
5. Preparação de um ambiente para aplicações de console

Java SE (Standard Edition)

- **OpenJDK (Open Java Development Kit):**
implementação de referência, livre e de código aberto, da plataforma Java, Standard Edition (Java SE). Desde o Java 7, o OpenJDK é a base sobre a qual o JDK da Oracle é construído. Na prática, para a maioria dos casos de uso, o OpenJDK e o Oracle JDK são funcionalmente idênticos, especialmente desde o Java 11, quando a Oracle alinhou ainda mais os dois projetos. Para o ambiente acadêmico, o OpenJDK é a escolha padrão e mais recomendada por ser **totalmente gratuito e livre de questões de licenciamento comercial**.

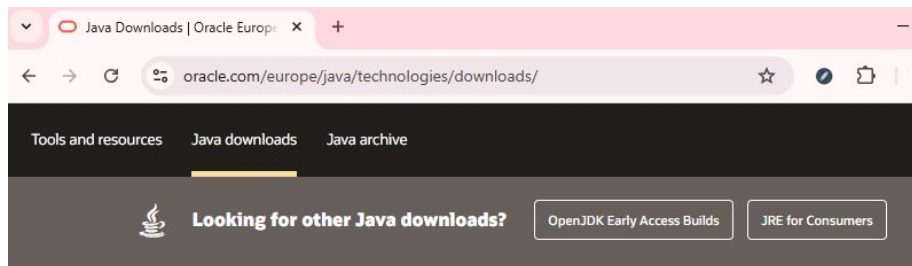
<https://openjdk.org/>



Java SE (Standard Edition)

- **JDK (comercial, como o da Oracle):**
A Oracle oferece seu próprio JDK, que até recentemente incluía alguns recursos comerciais e suporte de longo prazo (LTS) pago. Embora a Oracle tenha introduzido um modelo de licença mais permissivo (No-Fee Terms and Conditions - NFTC), para simplificar e evitar qualquer confusão de licenciamento no ambiente de ensino, o OpenJDK continua sendo a opção mais direta.

<https://www.oracle.com/europe/java/technologies/downloads/>



Java 24, Java 21, and earlier versions available now

JDK 24 is the latest release of the Java SE Platform.

[Learn about Java SE Subscription](#)

JDK 21 is the latest *Long-Term Support (LTS)* release of the Java SE Platform.

Earlier JDK versions are available below.

[JDK 24](#) [JDK 21](#) [GraalVM for JDK 24](#) [GraalVM for JDK 21](#)

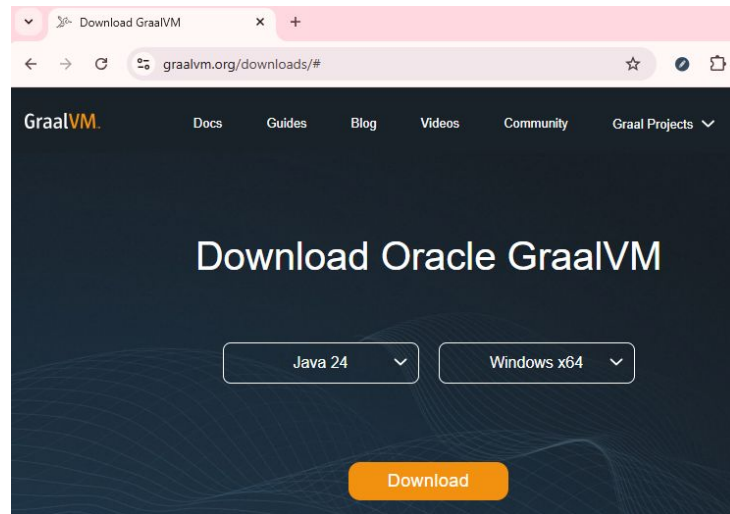
Java SE Development Kit 24.0.2 downloads

JDK 24 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and](#)

Java SE (standard Edition)

- **GraalVM (Graal Virtual Machine):** Lançado mais recentemente pela Oracle Labs, o GraalVM é uma extensão do JDK. Trata-se de uma JVM de alto desempenho e um JDK que não apenas otimiza a execução de código Java, mas também suporta a interoperabilidade com outras linguagens como JavaScript, Python e R. A grande inovação do GraalVM para o mundo Java é a Native Image. Essa tecnologia permite a compilação ahead-of-time (AOT) de uma aplicação Java em um executável nativo.

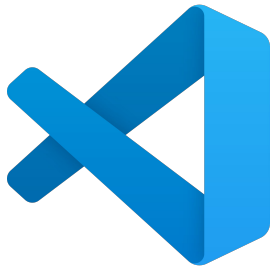
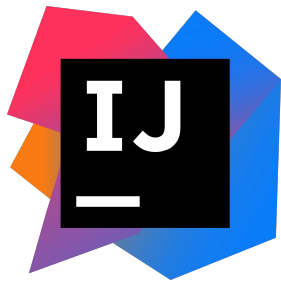
<https://www.graalvm.org/downloads>



Java SE (Standard Edition)

- **Outras distribuições**

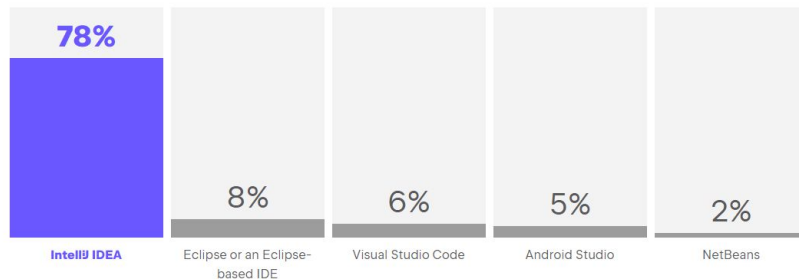
- Há várias outras distribuições do JDK baseadas no OpenJDK, como Amazon Corretto, AdoptOpenJDK, entre outras.
- Há até um build do OpenJDK da Microsoft para o Windows:
 - <https://learn.microsoft.com/pt-br/java/openjdk/download>
- Elas oferecem alternativas com diferentes características e licenças, geralmente adaptadas para plataformas específicas.



Editores de código e IDEs

jetbrains.com/lp/devecosystem-2023/java/

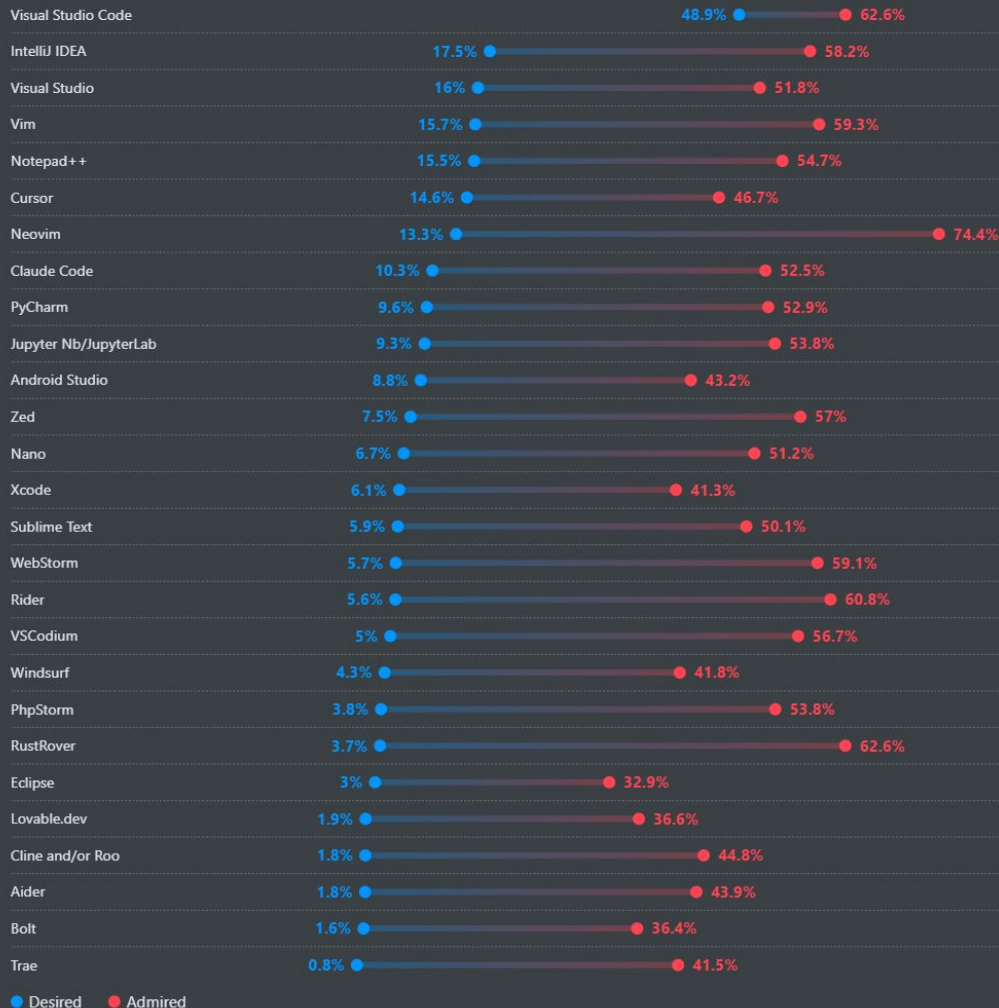
Which IDE / editor do you use the most for Java development?



Dev IDEs

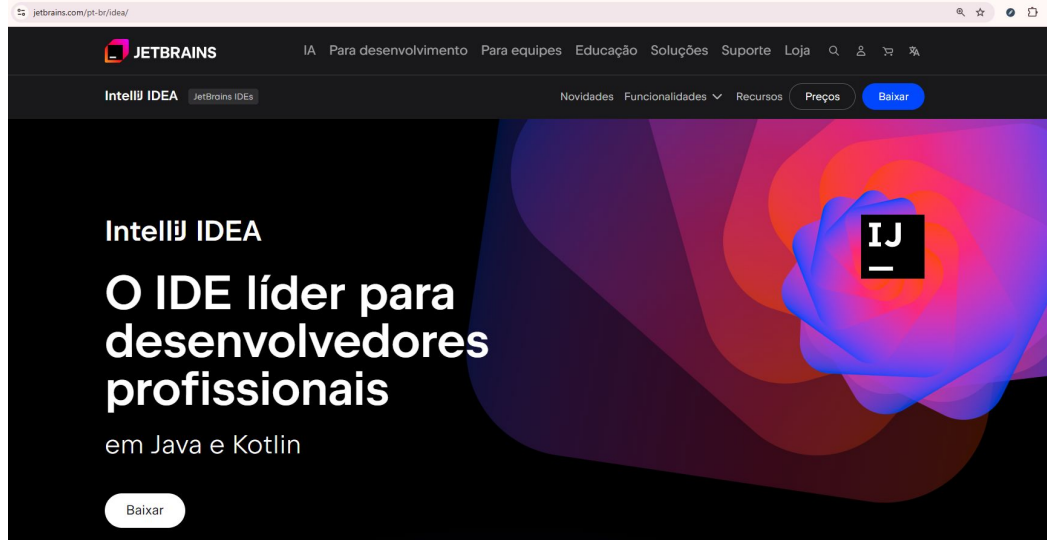
Not only is Visual Studio Code the most used IDE for five years in a row, it is also consistently ranked the most desired IDE. Developers that haven't used VS Code overwhelmingly want to use it in the next year.

? Which **development environments** and **AI-enabled code editing tools** did you use regularly over the past year, and which do you want to work with over the next year? Please check all that apply.



IntelliJ IDEA

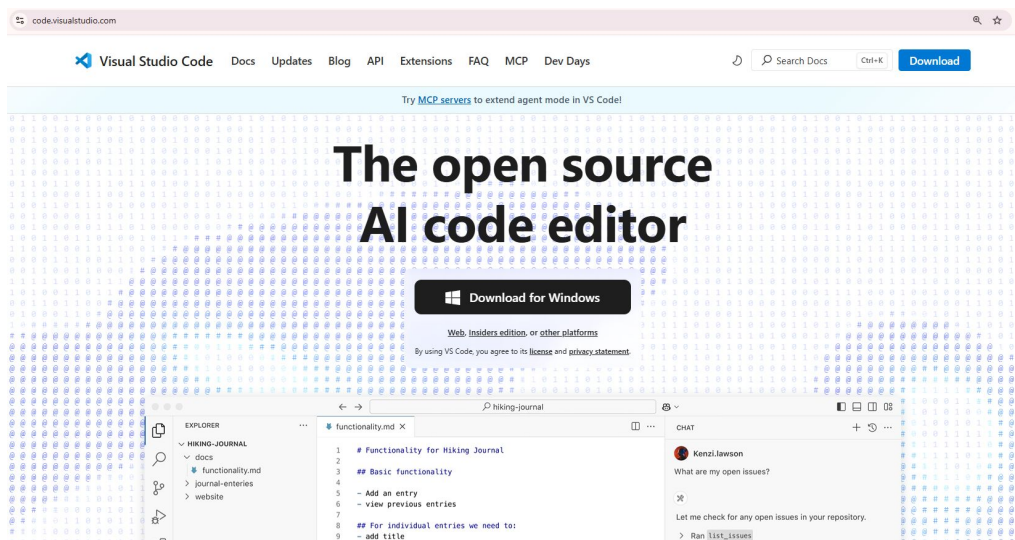
- Lançado pela **JetBrains** em **2001**, foi a primeira IDE Java a oferecer recursos avançados de refatoração de código e análise estática, que hoje são padrão de mercado.
- Por muitos anos, foi visto como uma alternativa paga e premium ao Eclipse. Com a **introdução da versão Community (gratuita e de código aberto)**, sua popularidade explodiu. Atualmente, é amplamente considerado o líder de mercado, famoso por sua inteligência de código, ergonomia, ferramentas de build integradas (Maven/Gradle) e um ecossistema de plugins robusto. A versão Ultimate (paga) adiciona suporte avançado para frameworks de backend (Spring, Jakarta EE) e frontend.
- É a **escolha predominante** no desenvolvimento profissional. Sua capacidade de entender o contexto do código ("code intelligence") é inigualável, o que acelera drasticamente o desenvolvimento e a manutenção.



<https://www.jetbrains.com/pt-br/idea>

Visual Studio Code

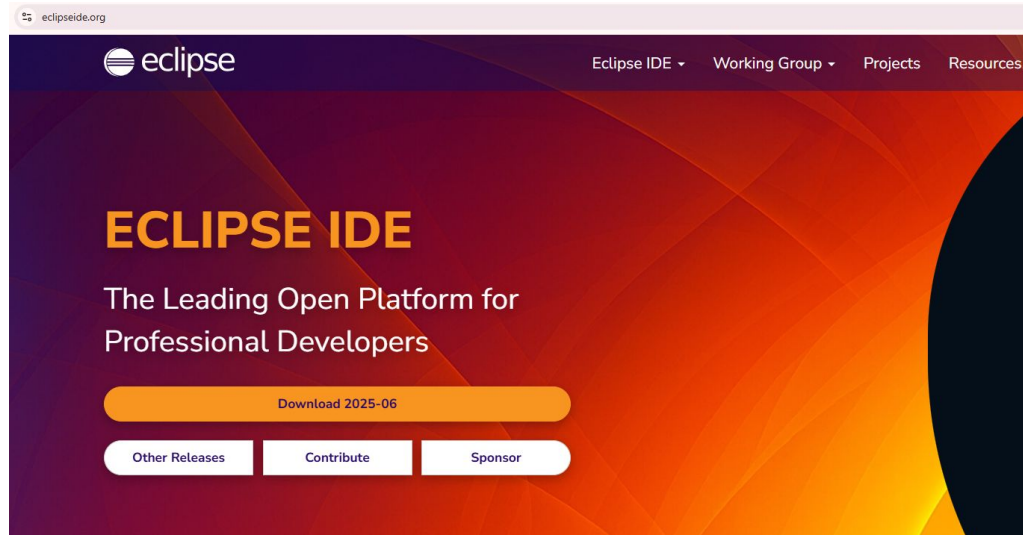
- Lançado pela **Microsoft** em **2015**, o VS Code não é uma IDE tradicional, mas um editor de código leve e extensível. Inicialmente focado em desenvolvimento web (JavaScript/TypeScript), ele rapidamente ganhou popularidade em diversas linguagens graças ao seu modelo de extensões. Com o "Extension Pack for Java" da Microsoft e da Red Hat, ele se tornou uma alternativa extremamente viável e popular para o desenvolvimento Java, especialmente para projetos baseados em Maven ou Gradle e para iniciantes.
- É a **escolha favorita de estudantes, desenvolvedores que trabalham com múltiplas linguagens** (polyglot programmers) e para quem prefere um **ambiente mais leve e rápido**. É excelente para o desenvolvimento de microsserviços e projetos com Spring Boot, Quarkus e Micronaut. Sua popularidade continua a crescer exponencialmente.



<https://code.visualstudio.com>

Eclipse IDE

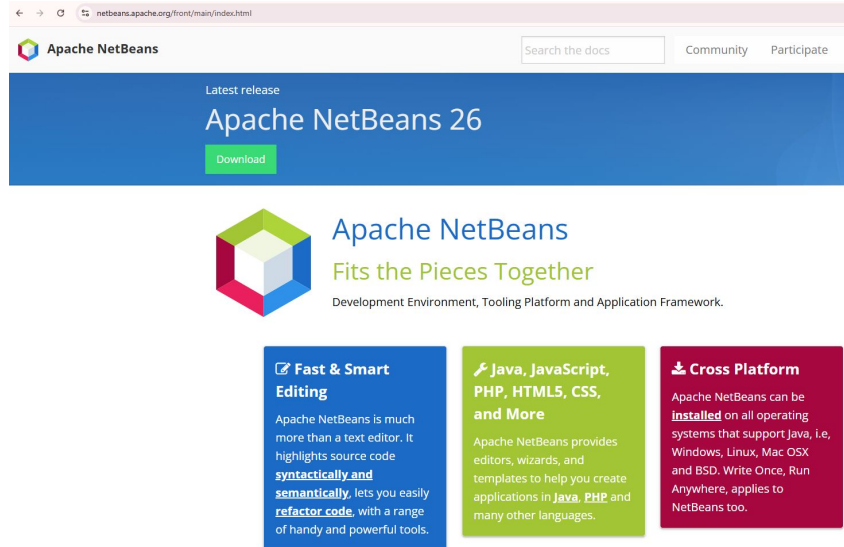
- Lançado pela **IBM** em **2001** e posteriormente gerenciado pela Eclipse Foundation, o Eclipse foi por muito tempo a IDE Java de código aberto dominante. Sua arquitetura baseada em plugins permitiu a criação de um ecossistema gigantesco para diversas finalidades. Foi fundamental na popularização do desenvolvimento Java profissional. Embora tenha perdido a liderança para o IntelliJ IDEA em termos de preferência, ainda é uma ferramenta poderosa, gratuita e muito utilizada, especialmente em projetos legados, no mundo acadêmico e em algumas grandes corporações.
- **Continua relevante** em empresas que padronizaram seu fluxo de trabalho nele há anos. Também é a base para outras ferramentas de nicho, como o Spring Tools Suite (STS) versão 3 e algumas IDEs para desenvolvimento Android (antes da migração para o IntelliJ/Android Studio).



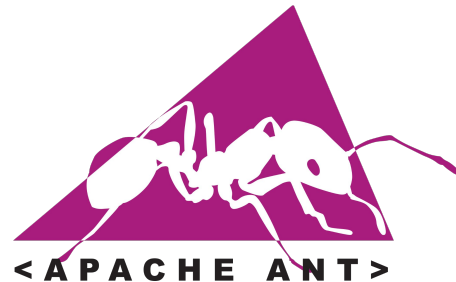
<https://eclipseide.org>

NetBeans

- Uma das IDEs Java mais antigas, com origem em um **projeto estudantil de 1996**, foi adquirida pela Sun Microsystems e se tornou a IDE "oficial" do Java por muitos anos. Era conhecida por suas excelentes ferramentas de desenvolvimento de interfaces gráficas (Swing). Em 2016, a Oracle doou o projeto para a Apache Software Foundation, o que revitalizou seu desenvolvimento. O Apache NetBeans continua sendo uma IDE de código aberto sólida, com bom suporte a Maven e Gradle e funcionalidades modernas.
- Possui uma base de usuários leal, mas sua participação de mercado é consideravelmente menor que a dos três líderes. É **frequentemente utilizado no ambiente acadêmico** devido à sua simplicidade e história ligada à própria Sun/Oracle.



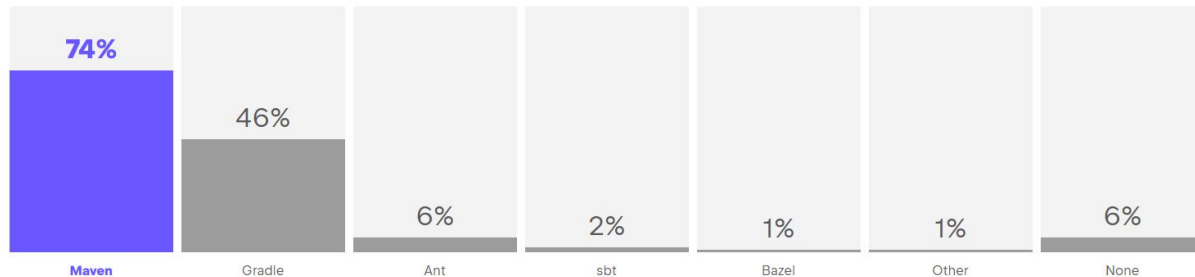
<https://netbeans.apache.org>



Gerenciadores de dependências

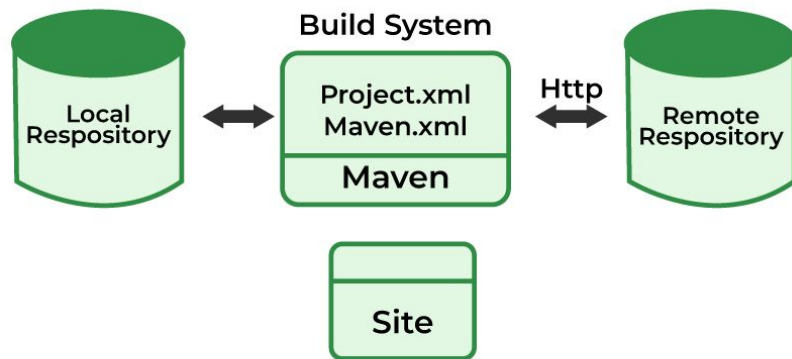
jetbrains.com/lp/devecosystem-2023/java/

Which build systems do you regularly use?



O que são e para que servem?

Os gerenciadores de dependências (ou ferramentas de build) são o pilar do desenvolvimento Java moderno. Eles automatizam o processo de baixar bibliotecas, compilar o código, executar testes e empacotar a aplicação, eliminando o caos que era o gerenciamento manual de arquivos `.jar`.



Apache Maven

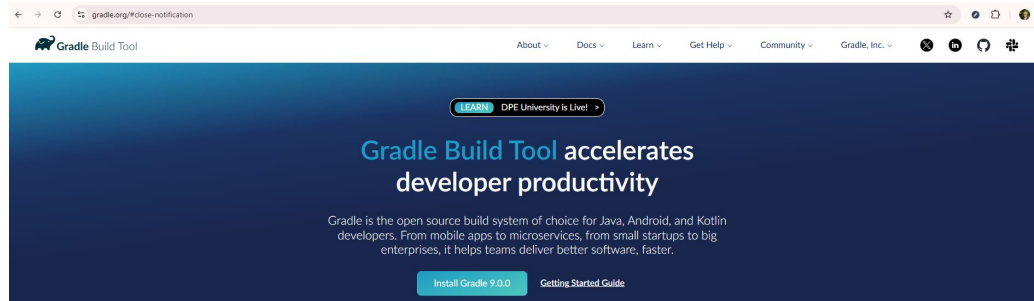


- Lançado em **2004** pela **Apache**, o Maven revolucionou o ecossistema Java ao introduzir duas ideias poderosas: o conceito de "convenção sobre configuração" e um repositório centralizado de dependências (o **Maven Central**). Antes dele, projetos usavam ferramentas como o Apache Ant, que exigia a escrita de scripts complexos e o gerenciamento manual de JARs. O Maven padronizou a estrutura de diretórios (*src/main/java*, *src/test/java*) e o ciclo de vida do build (*compile*, *test*, *package*, *etc.*). Seu arquivo de configuração declarativo, o **pom.xml** (Project Object Model), tornou a gestão de dependências e a construção de projetos drasticamente mais simples e replicáveis. Atualmente, o Maven é uma ferramenta extremamente madura, estável e ainda amplamente utilizada em todo o mercado.
- Por ter sido o padrão por tantos anos, o Maven é encontrado em uma vasta quantidade de projetos, desde sistemas legados até novos desenvolvimentos. É conhecido por sua robustez e pela enorme quantidade de plugins e integrações disponíveis. **Muitas empresas o utilizam como padrão para seus projetos Java.**
- Aproximadamente 62% de uso

<https://maven.apache.org>

Gradle

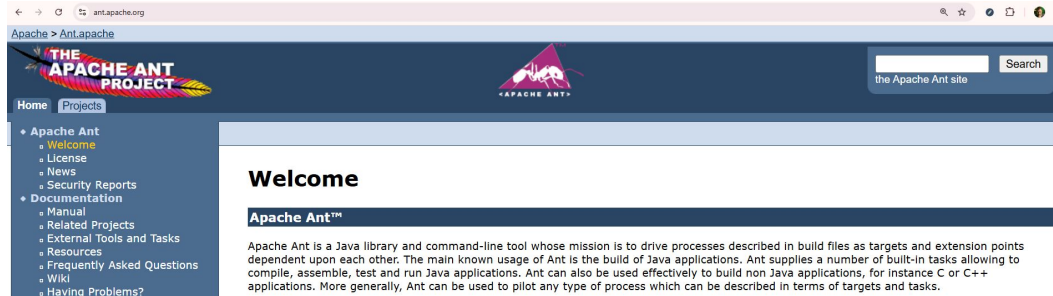
- Lançado em **2007** e ganhando tração significativa a partir de 2012, o Gradle foi criado para superar algumas das limitações do Maven, principalmente a rigidez do XML e a performance em projetos muito grandes. Em vez de XML, o Gradle utiliza uma DSL (Domain-Specific Language) baseada em Groovy ou Kotlin para seus scripts de build (*build.gradle*). Isso oferece uma flexibilidade e um poder de customização muito maiores. Suas principais vantagens são a performance superior (devido a caches avançados e daemons de compilação) e a expressividade de seus scripts de build. O Google escolheu o Gradle como a ferramenta de build padrão para o desenvolvimento Android, o que impulsionou massivamente sua adoção.
- Gradle é a **escolha predominante no desenvolvimento Android** e é muito popular em grandes projetos monolíticos ou com builds complexos, onde sua performance e flexibilidade fazem uma grande diferença. Também é amplamente utilizado em projetos de código aberto e por equipes que buscam o máximo de desempenho e customização em seu processo de build.



<https://gradle.org>

Apache Ant

- Lançado em **2000**, o Ant foi o predecessor do Maven. Ele é uma ferramenta de build baseada em scripts XML, mas não possui um sistema de gerenciamento de dependências nativo. Para gerenciar dependências com o Ant, era comum usar uma ferramenta complementar chamada Apache Ivy. O Ant oferece controle total sobre o processo de build, mas exige que o desenvolvedor defina explicitamente todas as tarefas (compilar, copiar arquivos, etc.), tornando-o muito mais verboso e menos padronizado que o Maven.
- Hoje em dia, o Ant **é considerado uma ferramenta legada**. É muito raro ver novos projetos começando com Ant. Sua presença se limita a sistemas mais antigos que ainda não foram migrados para Maven ou Gradle. Conhecê-lo é útil para arqueologia de software, mas não é uma habilidade demandada para novos desenvolvedores.





Agentes de Codificação - O que são

- Um **"agente codificador"** (ou **AI Coding Assistant**) é uma ferramenta de inteligência artificial projetada para auxiliar desenvolvedores diretamente no ambiente de programação. Ele atua como um "par programador" virtual, com a capacidade de:
 - **Autocompletar código:** De trechos simples a blocos complexos de lógica.
 - **Gerar código a partir de linguagem natural:** Você descreve o que quer em um comentário (ex: "// cria uma função para ler um arquivo CSV e retornar uma lista de produtos") e o agente escreve o código.
 - **Explicar e documentar código:** Selecionar um trecho de código e pedir uma explicação ou a geração de documentação.
 - **Encontrar bugs e sugerir correções:** Analisar o código em busca de erros lógicos ou de sintaxe.
 - **Atuar como um "chat" de conhecimento:** Fazer perguntas sobre algoritmos, bibliotecas ou boas práticas diretamente no terminal ou IDE.
- Essencialmente, ele integra o poder de um Modelo de Linguagem Grande (LLM) como o Gemini ao fluxo de trabalho do desenvolvedor.



Agentes de Codificação - História

- A ideia de assistência de código não é nova, mas a tecnologia evoluiu drasticamente.
 - **Primórdios (Anos 90-2000):** O início foi marcado por ferramentas de autocompletar baseadas em análise estática (ex: IntelliSense da Microsoft). Elas eram ótimas para sugerir nomes de métodos e variáveis, mas não entendiam a intenção do programador.
 - **Primeira Onda de IA (Anos 2010):** Ferramentas como o Kite e o Tabnine (em suas versões iniciais) começaram a usar modelos de Machine Learning mais simples para prever sequências de código com base em padrões aprendidos de milhões de projetos de código aberto. As sugestões se tornaram mais contextuais e inteligentes.
 - **A Revolução dos LLMs (2021-Presente):** O lançamento do GitHub Copilot, baseado no modelo GPT da OpenAI, mudou o jogo. Pela primeira vez, uma ferramenta podia entender linguagem natural e gerar blocos inteiros de código funcional. Isso marcou o início da era dos agentes codificadores modernos, que foram rapidamente seguidos por concorrentes como Cursor, WindSurf, Claude, ChatGPT e, claro, o Gemini do Google.



CURSOR



GEMINI CLI

🎯 **Free tier:** 60 requisições/min e 1,000 requisições/dia com conta pessoal do Google

🧠 **Powerful Gemini 3.1 Pro:** 1M de tokens na janela de contexto

🔧 **Built-in tools:** Google Search grounding, manipulação de arquivos, shell commands, web fetching

🔌 **Extensible:** Suporte ao MCP (Model Context Protocol) para integrações personalizadas

💻 **Terminal-first:** Projetado para linha de comando, funciona com qualquer editor ou IDE

🛡️ **Open source:** Apache 2.0 licensed

<https://github.com/google-gemini/gemini-cli>



Preparação do ambiente no VSCode

Preparação do ambiente no VSCode

Passo 0: Downloads Prévios

Antes de começar, baixe os seguintes programas. Apenas baixe, não precisa instalar ainda.

1. **Visual Studio Code:** O editor de código:
 - a. <https://code.visualstudio.com>
2. **Node.js:** O Gemini CLI precisa do Node.js para funcionar. Baixe a versão LTS (Long-Term Support):
 - a. <https://nodejs.org>

Preparação do ambiente no VSCode

Passo 1: Instalação das Ferramentas Base

1. **Instale o Visual Studio Code:** Use o instalador que você baixou. Aceite as opções padrão.
2. **Instale o Node.js:** Use o instalador baixado. Aceite as opções padrão.
3. **Instale o Gemini CLI:**
 - Abra o menu Iniciar e procure por "Command Prompt" ou "PowerShell" (no Windows) ou abra o "Terminal" (no macOS/Linux).
 - Digite o seguinte comando e pressione Enter:
npm install -g @google/gemini-cli
4. **Instale o plugin Gemini Code Assist:**
5. **Configure sua Chave de API do Gemini:**
 - Acesse o Google AI Studio:
<https://aistudio.google.com/app/apikey>
 - Clique em "**Create API key in new project**". Copie a chave gerada.
 - No terminal, execute o comando abaixo, substituindo **SUA_CHAVE_AQUI** pela chave que você copiou:
gemini auth SUA_CHAVE_AQUI
 - Alternativamente, você pode usar o GEMINI CLI apenas autenticando-se com Google

Preparação do ambiente no VSCode

Passo 2: Conversando com o Gemini para Montar o Ambiente

Agora, vamos usar o terminal para pedir ajuda ao Gemini. Esta é uma simulação de como você pode usar a ferramenta para pedir orientação e resolver dúvidas.

Aluno 🧑 : Abre o terminal e digite:

gemini "Olá, Gemini! Sou um estudante iniciante em Java. Acabei de instalar o VS Code e o seu CLI. O que mais eu preciso instalar no meu computador para criar e rodar um programa simples em Java que funcione no console usando os gerenciadores de pacotes Maven e Graddle? Me explique o que é cada coisa e por que eu preciso delas."

Siga os passos, com cuidado. Analise criticamente e questione se não entender ou quiser se aprofundar sobre algo. Se quiser agilizar o processo, pode fazer download do [maven](#) e do [graddle](#). O JDK, ele irá sugerir de acordo com o sistema operacional e contexto. Se for o windows, recomendo o [build do OpenJDK da Microsoft](#).

Preparação do ambiente no VSCode

Passo 3: Criando e Executando o "Olá, Mundo!" com Gemini

Vamos pedir ao Gemini o passo a passo para criar e rodar o projeto.

Aluno :

gemini "Agora, me dê os comandos exatos que eu preciso usar no terminal para criar um projeto Java 'Olá, Mundo!'. Depois, me diga como compilar e executar esse projeto."

Siga os passos, com cuidado. Analise criticamente e questione se não entender ou quiser se aprofundar sobre algo.

Vídeo de demonstração

